

Practical: Git Repository Management

Aim

Create a Git repository, initialize it, and add a Python project.

Software Required

- Git for Windows (already installed)
 - Visual Studio Code
 - Python (installed)
 - GitHub account (optional for Part A)
-

Folder Structure

You will create this folder:

```
Git_Practical
├── hello.py
├── README.md
└── .git
```

Step 1: Open Visual Studio Code

1. Click **Start Menu**.
 2. Search for **Visual Studio Code**.
 3. Open it.
-

Step 2: Open the Terminal

In VS Code,

Click

Terminal



New Terminal

A terminal opens at the bottom.

It looks like

```
PS C:\Users\YourName>
```

(PS means PowerShell.)

Step 3: Go to Desktop

Type

```
cd Desktop
```

Press **Enter**

Now you are inside the Desktop folder.

Example

```
PS C:\Users\Nikhita\Desktop>
```

Step 4: Create a Folder

Type

```
mkdir Git_Practical
```

Press Enter.

Nothing will happen on the screen.

It simply creates the folder.

Step 5: Move into the Folder

Type

```
cd Git_Practical
```

Press Enter.

Now terminal becomes

PS C:\Users\Nikhita\Desktop\Git_Practical>

Step 6: Open this Folder in VS Code

Type

code .

Press Enter.

The folder opens inside VS Code.

You should now see

Git_Practical

on the left Explorer.

Step 7: Create Python File

Click

New File

Name it

hello.py

Step 8: Write Python Program

Inside **hello.py**

```
print("Hello Git!")
```

Save

Ctrl + S

Step 9: Create README File

Again click

New File

Name it

README.md

Write

Git Practical

This is my first Git repository.

Project Name: Python Demo

Created by: Your Name

Save it.

Step 10: Check Folder

Your folder now contains

Git_Practical

hello.py

README.md

Step 11: Initialize Git Repository

Go to Terminal.

Type

git init

Press Enter.

Output

Initialized empty Git repository in
C:/Users/Nikhita/Desktop/Git_Practical/.git/

Now Git creates a hidden folder

.git

This folder stores all version history.

Step 12: Check Git Status

Type

git status

Output

On branch master

No commits yet

Untracked files:

hello.py
README.md

Meaning

Git has found two files.

But they are **not being tracked**.

Step 13: Add Files

To add all files

git add .

Press Enter.

OR

Add only one file

```
git add hello.py
```

Step 14: Check Status Again

Type

```
git status
```

Output

Changes to be committed:

new file: hello.py

new file: README.md

Now Git is ready to save these files.

Step 15: Configure Git (First Time Only)

If this is your first Git project, configure your identity.

Type

```
git config --global user.name "Your Name"
```

Example

```
git config --global user.name "Nikhita"
```

Press Enter.

Now email

```
git config --global user.email "your@email.com"
```

Example

```
git config --global user.email "nikhita@gmail.com"
```

Press Enter.

Step 16: Commit the Project

Now save the project permanently.

Type

```
git commit -m "Initial Python Project"
```

Press Enter.

Example output

```
[master (root-commit) abc1234]
```

```
2 files changed
```

```
create mode 100644 hello.py
```

```
create mode 100644 README.md
```

Congratulations!

Your first Git commit is complete.

Step 17: Check Status

Type

```
git status
```

Output

```
On branch master
```

```
nothing to commit, working tree clean
```

This means:

-  All changes are saved.
-  Repository is up to date.
-  There are no pending changes.

Step 18: View Commit History

Type

`git log`

Output

commit a12bc34d56...

Author: Nikhita

Date: ...

Initial Python Project

This shows the commit history.

Step 19: Modify the Python File

Open

hello.py

Change it to

```
print("Hello Git!")  
print("Welcome to Git Repository")
```

Save the file.

Step 20: Check Status

Type

`git status`

Output

modified: hello.py

Git detects that the file has changed.

Step 21: Save the Changes

Stage the modified file:

```
git add hello.py
```

Commit the changes:

```
git commit -m "Updated hello.py"
```

Step 22: View All Commits

Type

```
git log
```

Now you'll see two commits:

Updated hello.py

Initial Python Project

Final Folder Structure

Git_Practical

```
|
|— .git
|— hello.py
|— README.md
```

Commands Summary

Step	Command	Purpose
1	cd Desktop	Go to Desktop
2	mkdir Git_Practical	Create project folder
3	cd Git_Practical	Open the folder
4	code .	Open folder in VS Code
5	git init	Initialize Git repository

Step	Command	Purpose
6	git status	Check repository status
7	git add .	Stage all files
8	git commit -m "Initial Python Project"	Save the first version
9	git log	View commit history
10	git add hello.py	Stage modified file
11	git commit -m "Updated hello.py"	Save changes
12	git status	Verify working tree is clean

Viva Questions (with Answers)

Q1. What is Git?

Answer: Git is a distributed version control system used to track changes in files and manage source code during software development.

Q2. What is a Git repository?

Answer: A Git repository is a folder that contains project files along with a hidden .git directory where Git stores version history and metadata.

Q3. What does git init do?

Answer: It initializes a new Git repository by creating a hidden .git folder in the project directory.

Q4. What is the purpose of git add?

Answer: It stages new or modified files so they are ready to be committed.

Q5. What does git commit do?

Answer: It saves a snapshot of the staged changes in the repository with a descriptive message.

This completes **Part (a): Create a Git repository, initialize it, and add a Python project** in a detailed, beginner-friendly manner. Once you're comfortable with this, you can proceed to **Part (b): Branching, merging, and pull requests**.